



Python Programming Fundamentals

Function Basics

Programming Fundamentals Team

SETU

2026-09-14



Outline

1. What is a Function?
2. The def Keyword
3. Calling a Function
4. Void Functions and None
5. Docstrings
6. Functions in Practice
7. Building a Program with Functions



Outline

- 1. What is a Function?**
2. The def Keyword
3. Calling a Function
4. Void Functions and None
5. Docstrings
6. Functions in Practice
7. Building a Program with Functions



1. What is a Function?

A **function** is a named, reusable block of code that performs a specific task.



1. What is a Function?

A **function** is a named, reusable block of code that performs a specific task.

Why use functions?

- **Reuse** — write once, call many times



1. What is a Function?

A **function** is a named, reusable block of code that performs a specific task.

Why use functions?

- **Reuse** — write once, call many times
- **Abstraction** — hide complexity behind a simple name



1. What is a Function?

A **function** is a named, reusable block of code that performs a specific task.

Why use functions?

- **Reuse** — write once, call many times
- **Abstraction** — hide complexity behind a simple name
- **Organisation** — break a big program into small, understandable parts



1. What is a Function?

A **function** is a named, reusable block of code that performs a specific task.

Why use functions?

- **Reuse** — write once, call many times
- **Abstraction** — hide complexity behind a simple name
- **Organisation** — break a big program into small, understandable parts
- **Testing** — test each piece independently



1. What is a Function?

A **function** is a named, reusable block of code that performs a specific task.

Why use functions?

- **Reuse** — write once, call many times
- **Abstraction** — hide complexity behind a simple name
- **Organisation** — break a big program into small, understandable parts
- **Testing** — test each piece independently
- **Maintenance** — fix a bug in one place, fixed everywhere



1. What is a Function?

A **function** is a named, reusable block of code that performs a specific task.

Why use functions?

- **Reuse** — write once, call many times
- **Abstraction** — hide complexity behind a simple name
- **Organisation** — break a big program into small, understandable parts
- **Testing** — test each piece independently
- **Maintenance** — fix a bug in one place, fixed everywhere

You have already used built-in functions: `print()`, `input()`, `float()`, `int()`.

Now you will write your **own**.



Outline

1. What is a Function?
- 2. The `def` Keyword**
3. Calling a Function
4. Void Functions and None
5. Docstrings
6. Functions in Practice
7. Building a Program with Functions



2. The def Keyword

```
def function_name(parameters):  
    """Optional docstring."""  
    # function body  
    return value    # optional
```



2. The def Keyword

```
def function_name(parameters):  
    """Optional docstring."""  
    # function body  
    return value    # optional
```

NOTE:

The return statement (when there) has two purposes:

- to return a value from the function
- to exit the function (wherever it appears - see later with **if statements**)



2. The def Keyword

Simplest possible function:

```
def say_hello():  
    print("Hello, World!")  
  
# Call the function  
say_hello()      # Hello, World!  
say_hello()      # Hello, World!  
say_hello()      # Hello, World!
```

The function is **defined** once with `def` and **called** as many times as you like.



Outline

1. What is a Function?
2. The def Keyword
- 3. Calling a Function**
4. Void Functions and None
5. Docstrings
6. Functions in Practice
7. Building a Program with Functions



3. Calling a Function

A function does nothing until it is **called** (invoked).

```
def greet():  
    print("Good morning!")  
    print("Welcome to Python.")  
  
# Nothing happens yet – we haven't called it  
  
greet()      # Now it runs  
greet()      # Runs again
```



3. Calling a Function

Functions can call other functions:

```
def draw_line():  
    print("-" * 30) # 30 hyphens  
  
def print_header(title):  
    draw_line()  
    print(title)  
    draw_line()  
  
print_header("Student Report") # prints ---- Student Report ----
```



Outline

1. What is a Function?
2. The def Keyword
3. Calling a Function
- 4. Void Functions and None**
5. Docstrings
6. Functions in Practice
7. Building a Program with Functions



4. Void Functions and None

A function that has no return statement implicitly returns the special value None.

```
def greet(name):  
    print(f"Hello, {name}!")  
  
result = greet("Alice")    # prints Hello, Alice!  
print(result)              # None  
print(type(result))        # <class 'NoneType'>
```



4. Void Functions and None

A function that has no return statement implicitly returns the special value None.

```
def greet(name):  
    print(f"Hello, {name}!")  
  
result = greet("Alice")      # prints Hello, Alice!  
print(result)                # None  
print(type(result))          # <class 'NoneType'>
```

NOTE: **None** represents “no value” in Python. It is **not** zero, not an empty string — it is the absence of a value.



4. Void Functions and None

Common mistake:

```
def add(a, b):  
    print(a + b)    # prints but does NOT return  
  
result = add(3, 4)    # prints 7  
doubled = result * 2    # TypeError: NoneType * int
```



Outline

1. What is a Function?
2. The def Keyword
3. Calling a Function
4. Void Functions and None
- 5. Docstrings**
6. Functions in Practice
7. Building a Program with Functions



5. Docstrings

A **docstring** is a string literal at the top of a function body. It documents what the function does.



5. Docstrings

```
:  
(part of the calculate.py file)  
:  
def calculate_area(width, height):  
    """Calculate and return the area of a rectangle.  
  
    Args:  
        width: The width of the rectangle (number).  
        height: The height of the rectangle (number).  
  
    Returns:  
        The area as a number (width * height).  
    """  
    return width * height
```



5. Docstrings

Access docstrings with `help()` or `.__doc__`:

Step 1 — go to the terminal, start the REPL and import the file:

```
$ python3                # loads the interpreter (REPL)
>>> import calculate     # imports the calculate.py file
```



5. Docstrings

Step 2 — print the docstring:

```
>>> print(calculate.calculate_area.__doc__)  
Calculate and return the area of a rectangle.
```

Args:

width: The width of the **rectangle** (number).
height: The height of the **rectangle** (number).

Returns:

The area as a **number** (width * height).



5. Docstrings

NOTE: Best practice: every function should have a docstring.



Outline

1. What is a Function?
2. The def Keyword
3. Calling a Function
4. Void Functions and None
5. Docstrings
- 6. Functions in Practice**
7. Building a Program with Functions



6. Functions in Practice

```
def celsius_to_fahrenheit(celsius):  
    """Convert Celsius to Fahrenheit."""  
    return (celsius * 9 / 5) + 32  
  
def fahrenheit_to_celsius(fahrenheit):  
    """Convert Fahrenheit to Celsius."""  
    return (fahrenheit - 32) * 5 / 9
```



6. Functions in Practice

```
# Use the functions
boiling_c = 100
boiling_f = celsius_to_fahrenheit(boiling_c)
print(f"{boiling_c}°C = {boiling_f}°F")      # 100°C = 212.0°F

freezing_f = 32
freezing_c = fahrenheit_to_celsius(freezing_f)
print(f"{freezing_f}°F = {freezing_c}°C")    # 32°F = 0.0°C
```



Outline

1. What is a Function?
2. The def Keyword
3. Calling a Function
4. Void Functions and None
5. Docstrings
6. Functions in Practice
- 7. Building a Program with Functions**



7. Building a Program with Functions

```
def get_student_data():  
    """Prompt user for student name and score."""  
    name = input("Student name: ")  
    score = float(input("Score (0-100): "))  
    return name, score
```



7. Building a Program with Functions



7. Building a Program with Functions

```
def get_student_data():  
    """Prompt user for student name and score."""  
    name = input("Student name: ")  
    score = float(input("Score (0-100): "))  
    return name, score
```

```
def calculate_grade(score):  
    """Return letter grade for a numeric score."""  
    if score >= 90: return "A"  
    if score >= 80: return "B"  
    if score >= 70: return "C"  
    if score >= 60: return "D"  
    return "F"
```



7. Building a Program with Functions



7. Building a Program with Functions

```
def get_student_data():  
    """Prompt user for student name and score."""  
    name = input("Student name: ")  
    score = float(input("Score (0-100): "))  
    return name, score
```

```
def calculate_grade(score):  
    """Return letter grade for a numeric score."""  
    if score >= 90: return "A"  
    if score >= 80: return "B"  
    if score >= 70: return "C"  
    if score >= 60: return "D"  
    return "F"
```

```
def print_result(name, score, grade):  
    """Print formatted student result."""  
    print(f"\n{name}: {score:.1f} → Grade {grade}")
```



7. Building a Program with Functions



7. Building a Program with Functions

```
def get_student_data():  
    """Prompt user for student name and score."""  
    name = input("Student name: ")  
    score = float(input("Score (0-100): "))  
    return name, score
```

```
def calculate_grade(score):  
    """Return letter grade for a numeric score."""  
    if score >= 90: return "A"  
    if score >= 80: return "B"  
    if score >= 70: return "C"  
    if score >= 60: return "D"  
    return "F"
```

```
def print_result(name, score, grade):  
    """Print formatted student result."""  
    print(f"\n{name}: {score:.1f} → Grade {grade}")
```



7. Building a Program with Functions

```
# Main program
name, score = get_student_data()
grade = calculate_grade(score)
print_result(name, score, grade)
```



Thanks for Watching - Any questions?

